

Desafio Técnico – Desenvolvedor(a) Full Stack

Contexto Geral

Título: Desafio Técnico – Desenvolvedor(a) Full Stack (Rails + React)

Tema: Gestão de Cursos Online Colaborativa (CourseSphere Full Stack)

Objetivo: desenvolver uma aplicação web simples de gestão de cursos e aulas, com backend em API (preferencialmente Rails) e frontend (preferencialmente React), avaliando organização de código, entendimento de CRUD, autenticação básica e consumo de APIs.

A stack sugerida é **Rails no backend** e **React no frontend**, mas o participante é livre para usar outras tecnologias web (Node, Laravel, etc). Recursos mais avançados (deploy, Docker, testes, etc.) contam como diferenciais.

Requisitos Funcionais (Mínimos)

1. Autenticação e Registro

- Tela de **registro** para criação de usuário, com campos de nome, email e senha.
- Tela de **login** com email e senha.
- Usuário consegue entrar e sair da aplicação (login/logout).
- Páginas que listam ou permitem editar cursos/aulas devem ser acessíveis apenas para usuários autenticados.
- Caso o fluxo de registro seja implementado apenas via seed/script, o README deve explicar claramente como criar usuários.

2. Entidades principais

Implemente no backend as entidades a seguir (nomes e campos podem variar, mas a ideia deve ser equivalente).

User

- Atributos mínimos:
 - `name` — obrigatório.
 - `email` — obrigatório, formato de email.
 - `password` — obrigatório.
- Regras:
 - Email deve ser único.
 - Apenas usuários autenticados podem acessar a área de cursos/aulas.

Course

- Atributos mínimos:
 - `name` — obrigatório, mínimo 3 caracteres.
 - `description` — opcional.
 - `start_date` — obrigatório.
 - `end_date` — obrigatório e deve ser igual ou posterior a `start_date`.
- Regras simples:
 - Todo curso pertence a um usuário “criador” (ex.: campo `creator_id` ou associação ``belongs_to :creator, class_name: "User"`).
 - Apenas o criador pode editar ou excluir o curso.

Lesson

- Atributos mínimos:
 - `title` — obrigatório, mínimo 3 caracteres.
 - `status` — valores: `"draft"` ou `"published"`.
 - `video_url` — opcional, mas se informado, deve parecer uma URL válida.
 - `course_id` — referência ao curso.
- Regras simples:
 - Toda aula pertence a um curso.
 - Apenas o criador do curso pode criar/editar/excluir aulas daquele curso (ou, se quiser simplificar, qualquer usuário autenticado pode criar/editar/excluir).

3. API Backend

- Implementar uma API REST simples para:
 - Registro e login de usuários.
 - CRUD de `courses` (index, show, create, update, delete).
 - CRUD de `lessons` (pelo menos create, index por curso e delete; update é desejável).
- As respostas devem ser em JSON.

- Pode ser uma API “stateless” com token simples ou uma aplicação Rails tradicional usando sessão; o importante é deixar claro no README como autenticar o frontend.

4. Frontend

Telas mínimas esperadas:

1. Tela de Registro

- Formulário com nome, email e senha, com validações básicas (campos obrigatórios, senha com tamanho mínimo razoável).
- Após registrar, o usuário pode ser redirecionado para o login ou logado automaticamente.

2. Tela de Login

- Formulário com email e senha, com validações básicas.
- Ao logar com sucesso, redirecionar para o dashboard.
- Bloquear rotas protegidas quando o usuário não estiver logado.

3. Dashboard / Lista de Cursos

- Exibir lista de cursos do usuário autenticado (ou todos, se preferir simplificar).
- Mostrar pelo menos `name` e datas.
- Botão para criar novo curso.

4. Detalhes do Curso

- Exibir:
 - `name`, `description`, `start_date`, `end_date`.
 - Lista de aulas (lessons) do curso.
- Ações:
 - Criar nova aula.
 - Editar/excluir curso (apenas se for o criador).
 - Editar/excluir aula (se implementar permissões, ótimo; se não, tudo bem deixar aberto).

5. Formulários

- Formulário de criação/edição de curso.
- Formulário de criação/edição de aula (pelo menos criação).
- Exibir mensagens de erro de validação básicas.

5. Consumo de API Externa

- Consumir **ao menos uma API pública externa** (por exemplo, RandomUser API ou outra de livre escolha).
- Usos possíveis:
 - Sugerir um “instructor convidado” para o curso, exibindo um nome/foto na tela de detalhes.
 - Mostrar avatares de usuários fictícios para ilustrar a turma.
- Não é obrigatório persistir esses dados no banco; é suficiente chamá-la no frontend e exibir o resultado.

6. Busca e Filtros (Simplificados)

Na listagem de cursos ou aulas, implemente **pelo menos um** dos itens:

- Campo de busca por nome de curso.
 - Filtro por status da aula (`draft` / `published`).
 - Paginação simples (pode ser feita no backend ou apenas frontend simulada).
-

Critérios de Avaliação

Prioridade Alta

- **Organização e estrutura do projeto:**
 - Separação clara entre backend e frontend (pastas, módulos, rotas).
 - Código legível, nomes de variáveis coerentes, arquivos não gigantes.
- **Modelagem básica e regras de negócio:**
 - Entidades e associações implementadas de forma consistente.
 - Regras simples (quem pode editar ou excluir) funcionando minimamente.

Prioridade Média

- **Qualidade visual e UX básica:**
 - Layout simples, mas organizado.
 - Responsividade básica.
- **Formulários e validações:**
 - Exibição de erros de validação vindos do backend ou do frontend.
- **Feedback de interação:**
 - Mostrar feedback de “carregando” ao buscar dados, ou mensagens simples de erro/sucesso.
- **Consumo de APIs:**
 - Integração correta com o backend (CRUD via HTTP) e com uma API externa.

Prioridade Baixa

- **Completeness:**
 - Todas as telas e regras descritas funcionam.
 - Detalhes como paginação, filtros combinados, etc.
-

Diferenciais (Nível Mais Avançado)

Todos os itens abaixo são opcionais e contam pontos extra.

- **Stack recomendada:**
 - Backend em **Ruby on Rails** (API mode ou aplicação tradicional) com boas práticas de MVC.
 - Frontend em **React** com hooks, organização mínima por componentes e rotas.
 - **Deploy funcional:**
 - Backend em algum serviço como Render, Fly.io, Railway, etc.
 - Frontend em Netlify, Vercel, GitHub Pages, etc.
 - README explicando URLs.
 - **Docker:**
 - `docker-compose` para subir backend + banco de dados (e frontend, se quiser).
 - **Testes automatizados:**
 - Testes de modelo e/ou requests no backend.
 - Testes de componente ou de integração simples no frontend.
 - **Autenticação mais robusta:**
 - Uso de JWT ou biblioteca de autenticação adequada, com proteção real de rotas.
 - **Melhorias de UX/UI:**
 - Design mais trabalhado, uso de biblioteca de componentes, tratamento elegante de estados de erro e loading.
 - **Organização de commits e Git:**
 - Commits com mensagens semânticas.
 - Branches de feature bem organizadas.
-

Entrega

- **Repositório público no GitHub** contendo:
 - Código do backend.
 - Código do frontend (no mesmo repo ou em repos separados, mas claramente linkados).
- **README** (obrigatório) contendo:

- Descrição breve do projeto.
 - Passo a passo para rodar o backend.
 - Passo a passo para rodar o frontend.
 - Informar usuário(s) e senha(s) de teste (caso não haja fluxo de registro completo).
 - Se houver deploy: links para frontend e backend, com instruções de login.
 - Se houver Docker: comandos para subir e acessar o sistema.
-
- **Prazo:** conforme definido no edital.